

Alpaka: What, Where & When?

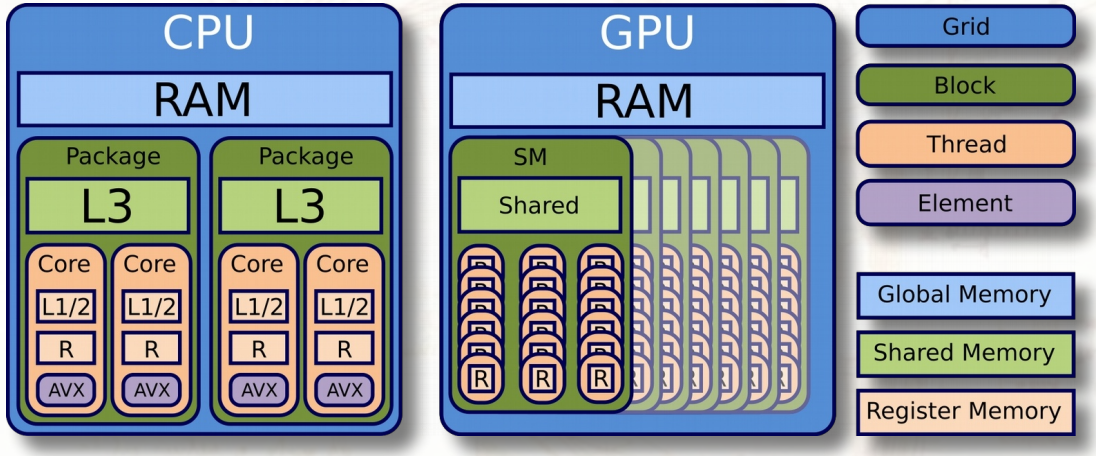
Tracker DPG workshop, 14-16 October 2026

Andrea Bocci

CERN EP-CMS-TDQ

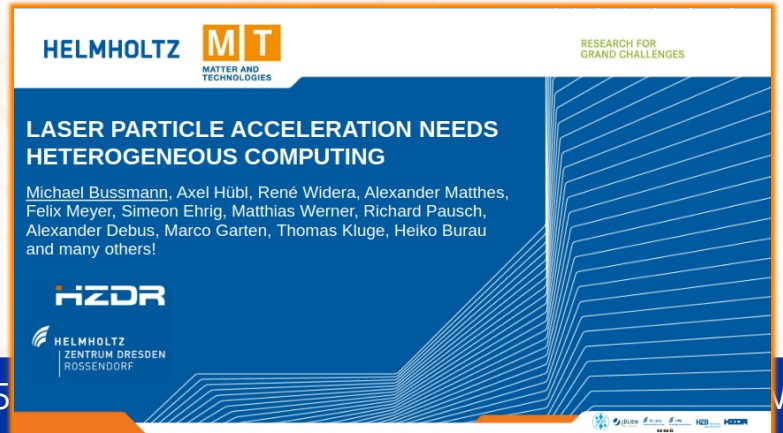
the alpaka performance portability library

- alpaka is a header-only library for heterogeneous software development and *performance portability*
  - written in C++20 for Linux, OS X, Windows
  - abstract the underlying levels of parallelism
  - expose the hardware details when necessary
  - *native performance* on different hardware
  - thread-safe, efficient for parallel applications
- supports all common hardware platforms
  - with serial and parallel execution on CPUs
    - c++ threads, TBB tasks, OpenMP
  - stable support for NVIDIA and AMD GPUs
    - with CUDA and ROCm backends
  - support for Intel GPUs and (experimental) Altera FPGAs, based on SYCL and oneAPI
- open source project, developed at HZDR and at CERN, with vibrant R&D activities
  - GitHub repository: https://github.com/alpaka-group/alpaka/
  - online documentation: https://alpaka.readthedocs.io/en/stable/



a short history of alpaka

- 2013-2015 HZDR team looks for a way to **extend PIConGPU to run on CPU**
  - none of the existing solution is satisfactory
  - Benjamin Worpitz starts the development of alpaka for his Master Thesis
  - René Widera develops upla as an alpaka-based drop in replacement for CUDA in PIConGPU
- July 2016: release of alpaka 0.1.0
- 2018 DLR starts using alpaka for the solver SPLISS
- 2019 alpaka presented at ACAT 2019 in Saas-Fee



- 2020 Berhard M. Gruber works on alpaka and llama as a doctoral student at CERN
- 2021 CERN starts evaluating and contributing to alpaka as the *performance portability* library for the CMS experiment
- November 2023: release of alpaka 1.0.0
  - support for Intel oneAPI, with SYCL Unified Shared Memory model
  - support c++23 `std::mdspan` and Kokkos' `mdspan`
- 2024 HZB starts using alpaka for RAYX
  - CMS migrates from CUDA to alpaka
  - alpaka^3 development starts
- June 2025: release of alpaka 2.0.0
  - move to c++20 and introduce Concepts
  - more CERN projects start using alpaka

CC BY SA

alpaka core concepts

Platform: a family of CPUs or GPUs

- host CPUs
- NVIDIA GPUs, AMD GPUs,
- SYCL platform

Device: one concrete CPU or GPU

- host CPU
- NVIDIA GPU, AMD GPU
- SYCL device

Queue: how work is scheduled on a device

- CPU thread, GPU stream, SYCL queue
- synchronous (blocking) or asynchronous

Events: is work along a queue done?

Task: function object to be executed

- host (callback) or device (kernel)
- named (class) or anonymous (lambda)

Accelerator: how a task is executed

- backend: CUDA, SYCL, OpenMP, TBB, …
- dimensionality: 1D, 2D, 3D, …
- blocks / threads / elements
- synchronisation, memory fences, atomic operations
- mathematical and warp functions

Buffer or **View**: memory on a device

- owning vs non-owning
- device
- dimensionality: 1D, 2D, 3D, …
- size and alignment

CC BY SA

alpaka features

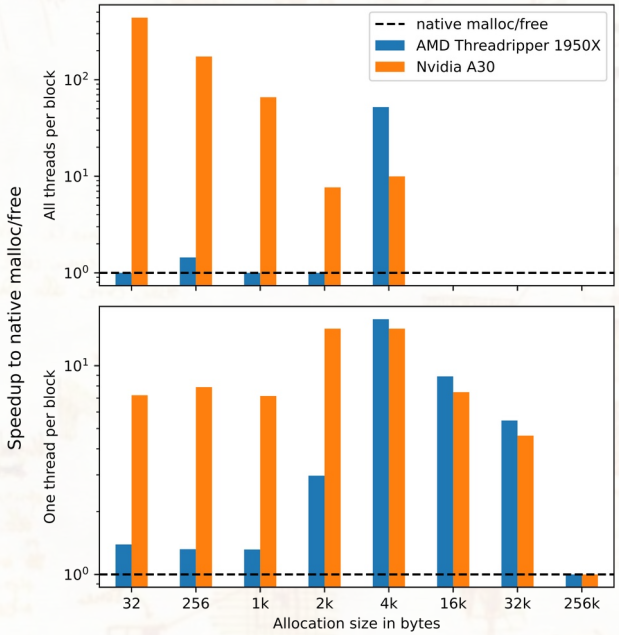


- **platforms** and **accelerators**
  - host CPU: serial, `std::thread`, OpenMP, TBB
  - NVIDIA GPUs: CUDA, SYCL
  - AMD GPUs: HIP/ROCm, SYCL
  - intel. GPUs: SYCL
  - altera. FPGAs: SYCL (Intel oneAPI 2025.0)
- memory **buffers**
  - host memory (pageable or pinned)
  - device memory
  - unified memory
- **tasks** and **kernels**
  - callbacks
  - static and dynamic shared memory
- **kernel** language
  - plain block / thread indices
  - N-dim strided loop over a block or grid
  - block and grid synchronisation
  - atomic operations and memory fences
  - warp-level primitives
- **math** functions
- **future developments**: execution graphs, SIMD and vectorisation, other accelerators

CC BY SA

mallocMC

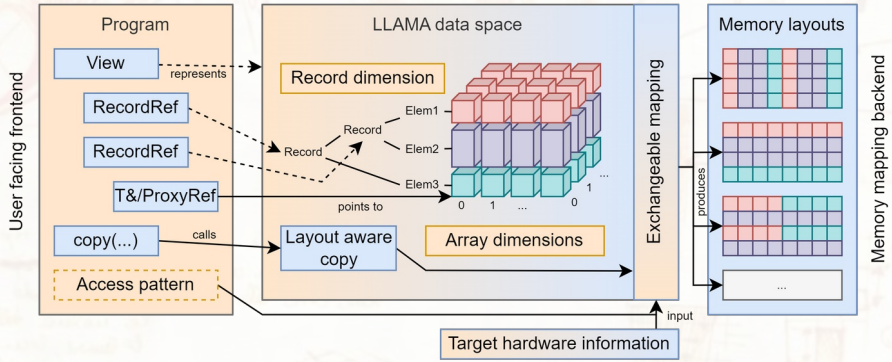
Blazingly fast many-core in-kernel allocation

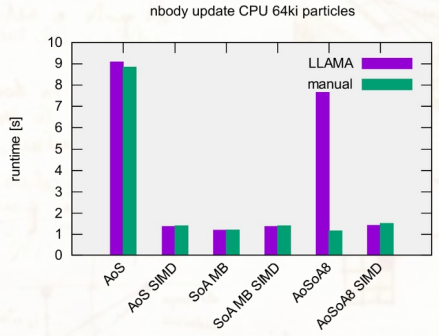


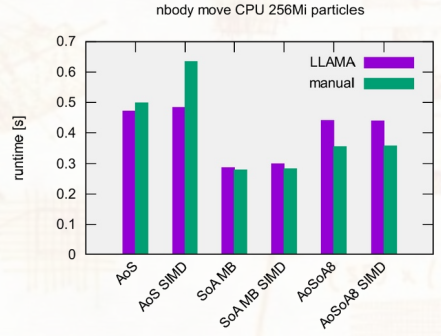
highly configurable framework for memory allocation algorithms

optimised for **small-chunk allocations in massively multi-threaded** scenarios (particularly GPUs)

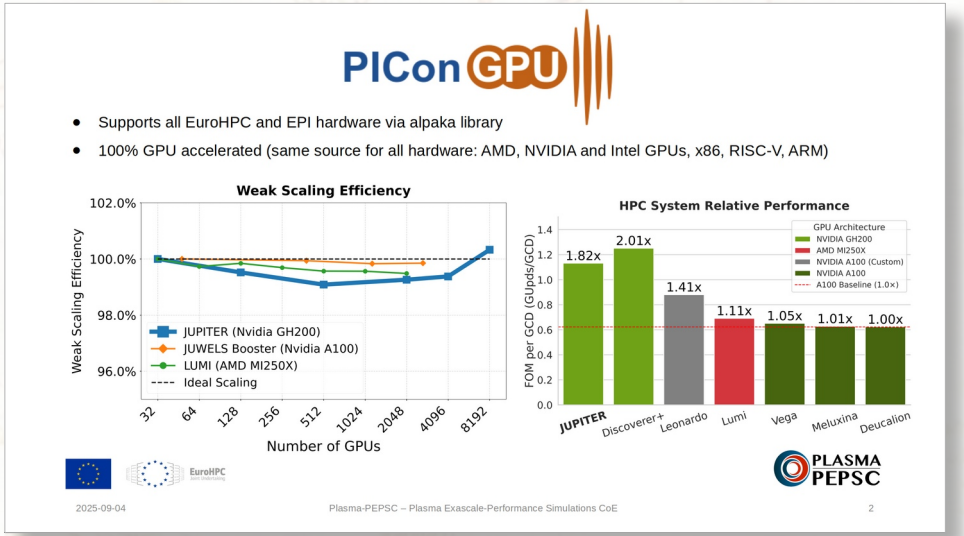
LLAMA Low-Level Abstraction of Memory Access library

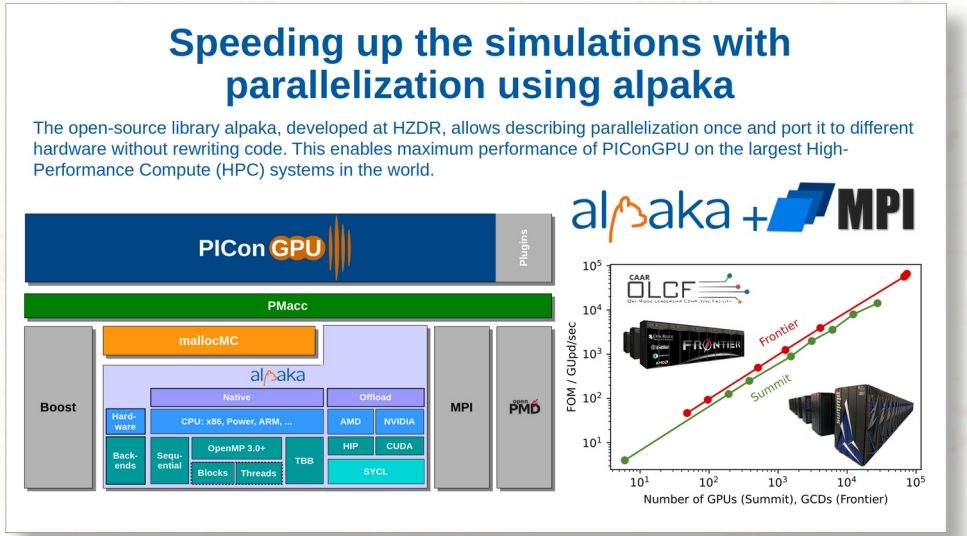






success stories





PIConGPU is a fully **relativistic Particle-in-Cell** (PIC) **code** for computational plasma physics, running on clusters of graphical processing units (GPUs)

- IEEE Transactions on Plasma Science – doi:10.1109/TPS.2010.2064310
- HZDR – www.hzdr.de/db/Cms?pOid=31887&pNid=3227
- GitHub – github.com/ComputationalRadiationPhysics/picongpu

HELMHOLTZ ZENTRUM DRESDEN ROSSENDORF

aerodynamics simulations

Deutsches Zentrum für Luft- und Raumfahrt
German Aerospace Center

Spliss

- library for approximate solution of linear equation systems with characteristics from aeronautical CFD
- focus on low-level performance and hardware technologies
- https://elib.dlr.de/217809/



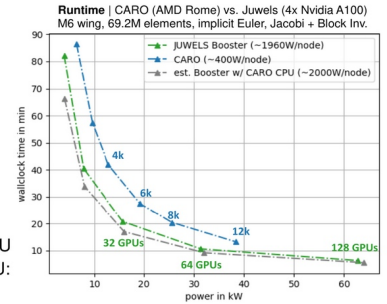
Heterogeneity: Support for Nvidia GPUs via Spliss

DLR

System – Juwels Booster
- AMD Epyc 7401 (2x 24 cores) per node
- 4x Nvidia Tesla A100 per node

System – CARO
- Nodes with 128 cores (AMD Rome, 2x CARA)
- 8 memory channels @3.2GHz (1.2x CARA)

Observations
- Node-wise comparison (“unfair”): 8-9x speedup
- Energy-wise comparison (“fair”): 1.6-1.9 speedup
- Performance limited by non-linear part on slow CPU
- Hypothetical Juwels Booster node with CARO CPU: 1.8-2.3 speedup (energy-wise)



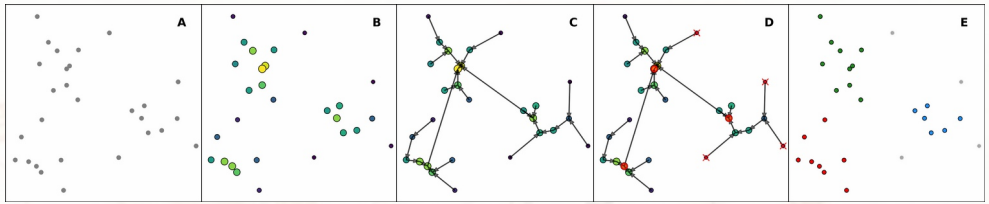
J. Mohnke and M. Wagner: A Look at Performance and Scalability of the GPU Accelerated Sparse Linear System Solver Spliss. In: Euro-Par 2023: Parallel Processing. DOI 10.1007/978-3-031-39698-4_43

Arne Rempke, German Aerospace Center (DLR), Institute of Software Methods for Product Virtualization, 40th WSSP, October 15th, 2025

“Using alpaka allows us to write a single Kernel to be executed on CPU or GPU”
Arne Rempke, DLR

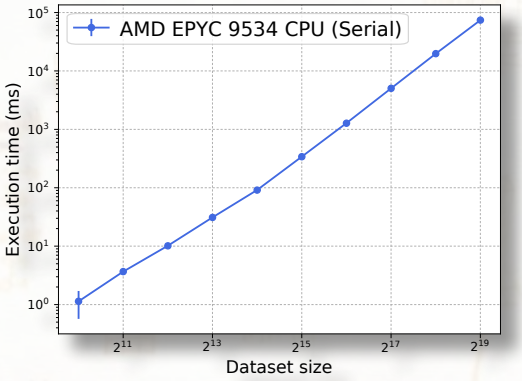
CC BY SA

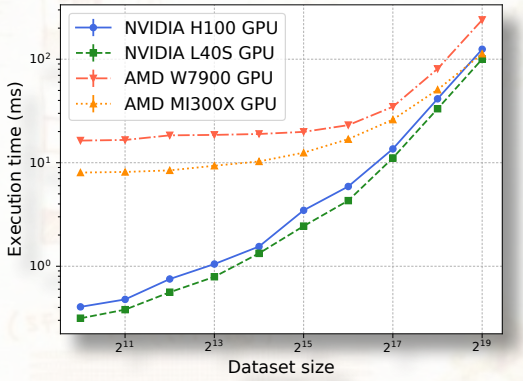
stering



CLUEstering is general purpose, N-dimensional density-based clustering algorithm

- offers C++ and python interfaces
- heterogeneous implementation based on alpaka (hidden to the user code)



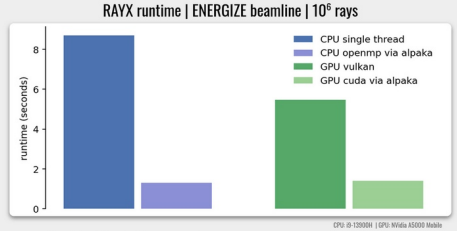


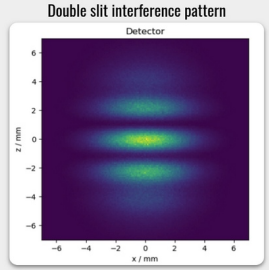
HZB rayx

HZB BESSY II Light Source

RAYX Optics simulation software for synchrotron applications

- Fast
- Non-sequential ray tracing
- Comes with optical elements with different curvatures
- Diffraction at slit interfaces
- Interference patterns by sum of electric field vectors

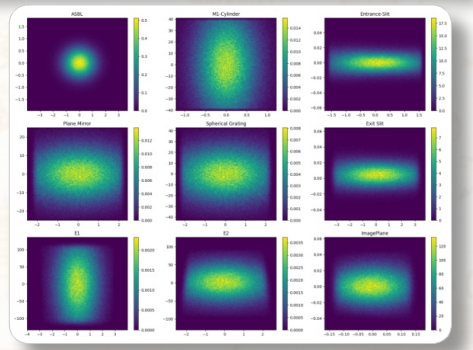




RAYX from Python

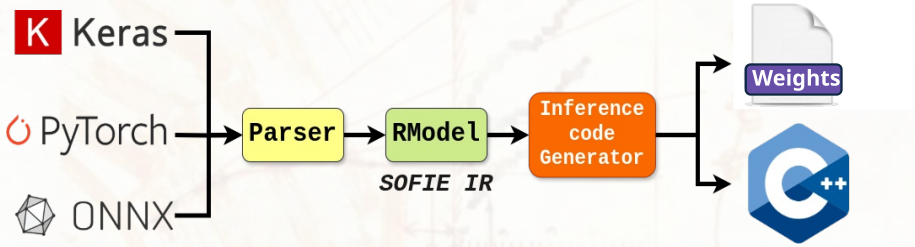
```
$ pip install rayx
```

github.com/hz-b/rayx

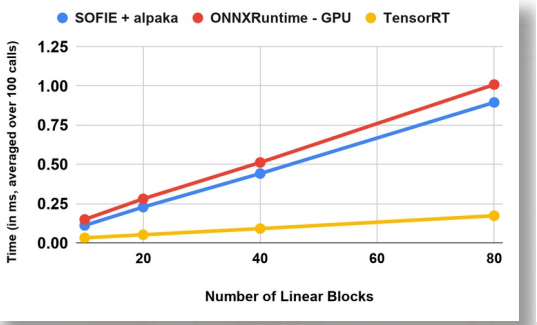


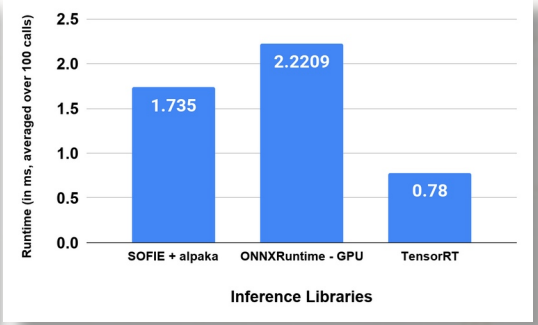
SOFIE

System for Optimized Fast Inference code Emit



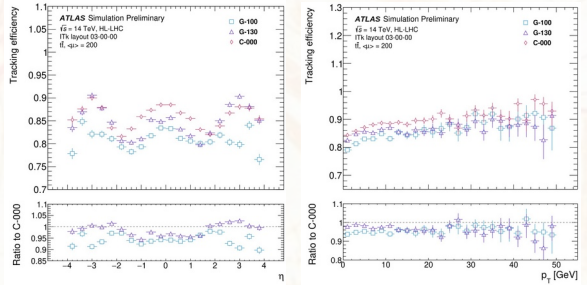
Inference on **heterogeneous architectures** with alpaka





NVIDIA RTX 2070 preliminary results

- **ACTS** is an experiment-independent track reconstruction software
  - R&D platform for algorithms and technologies



High-Luminosity **ATLAS** online reconstruction

  - traccc: a massively parallel reconstruction software

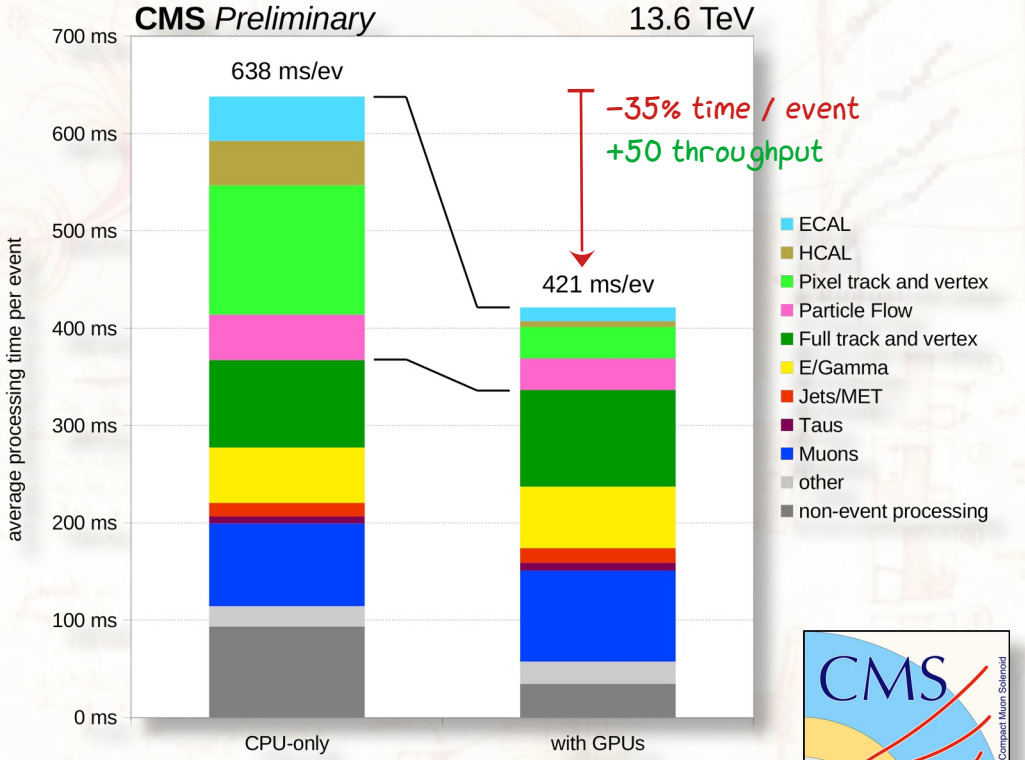
✅: exists, 🟡: work started, ⚪: work not started yet

Category	Algorithms	CPU	CUDA	SYCL	alpaka
Clusterization	CCL / FastSv / etc.	✅	✅	✅	✅
	Measurement creation	✅	✅	✅	✅
Seeding	Spacepoint formation	✅	✅	✅	✅
	Spacepoint binning	✅	✅	✅	✅
	Seed finding	✅	✅	✅	✅
	Track param estimation	✅	✅	✅	✅
Track finding	Combinatorial KF	✅	✅	✅	✅
Ambiguity resolution	Greedy resolver	✅	🟡	⚪	⚪
Track fitting	KF	✅	✅	🟡	✅

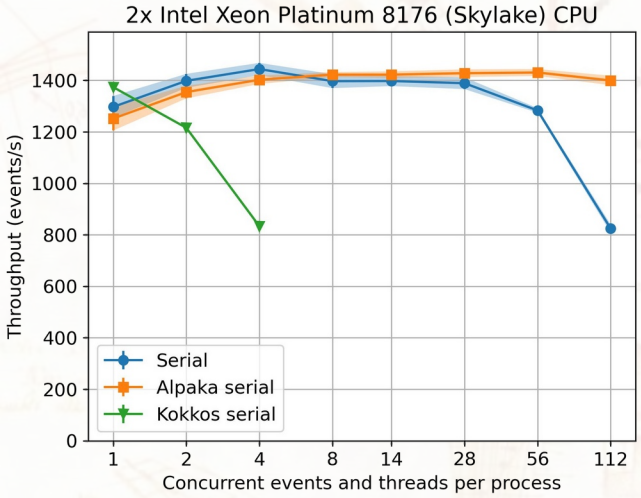
CC BY SA

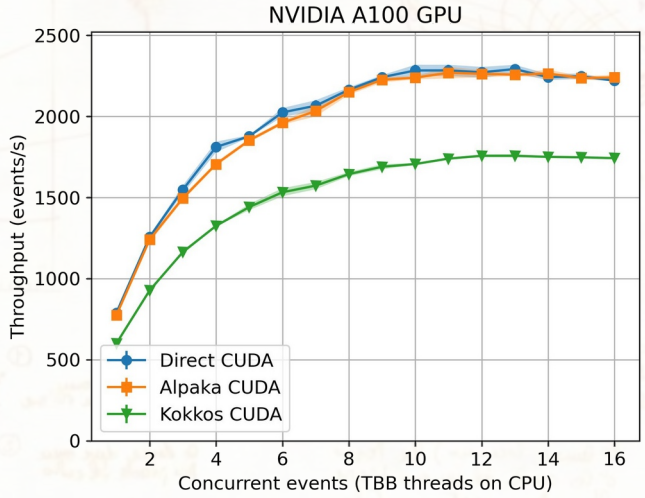
CMS experiment at CERN

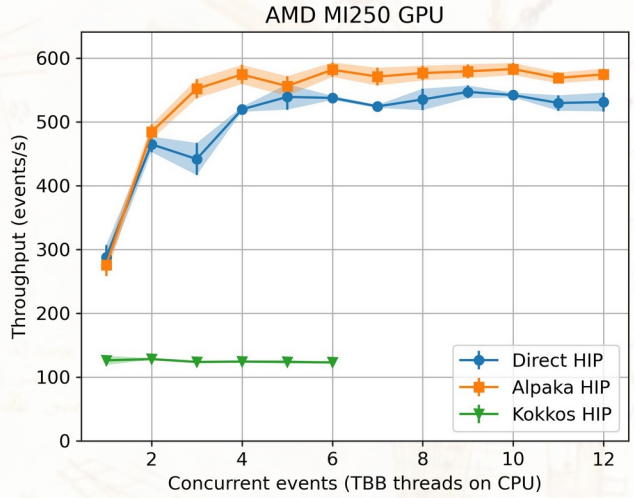
- CMS deployed the High Level Trigger on a heterogeneous computer farm in 2022
  - 200 nodes
  - 2 × AMD EPYC “Milan” 7763 processors
  - 2 × NVIDIA Tesla T4 GPUs
- thanks to the use of GPUs
  - 50% better event processing throughput
  - 15% - 20% better performance at initial cost
  - 15% - 25% better performance per kW
- by 2024 all code was migrated to alpaka
  - support for CPU and non-NVIDIA GPUs
  - simplify further development and validation



alpaka as the portability choice for CMS





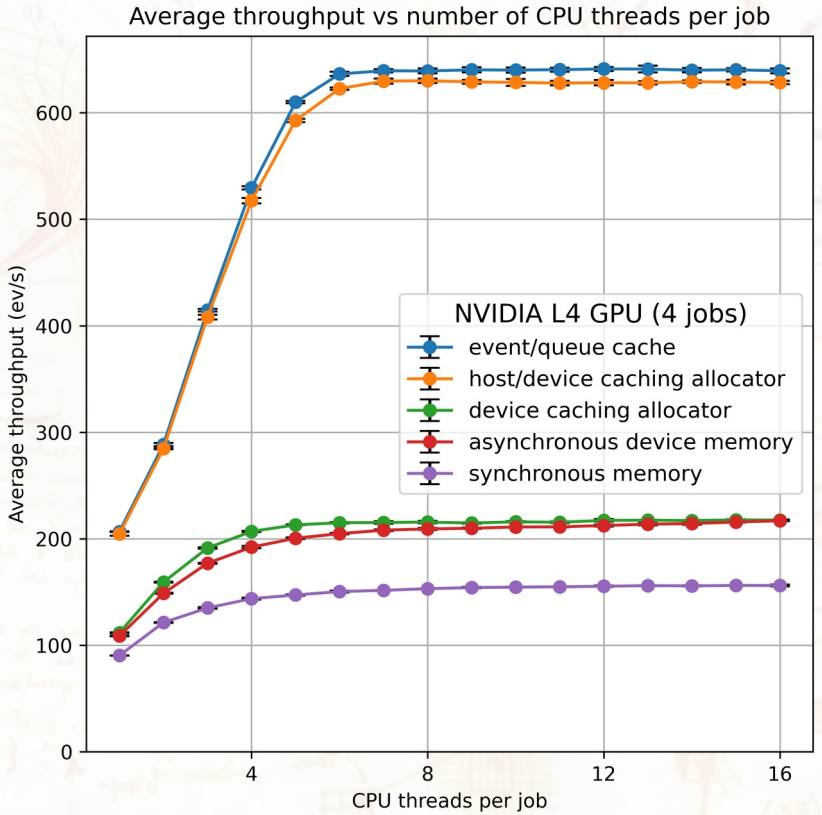


Patatrack pixel-only reconstruction, studies done at **CERN** and ***HEP-CCE***

- R&D activities in 2020-2022 to compare different options
  - evaluated using as a benchmark the Patatrack demonstrators
  - standard C++, CUDA, HIP, OpenMP, alpaka, kokkos, SYCL, …

CMSSW optimisations on top of alpaka

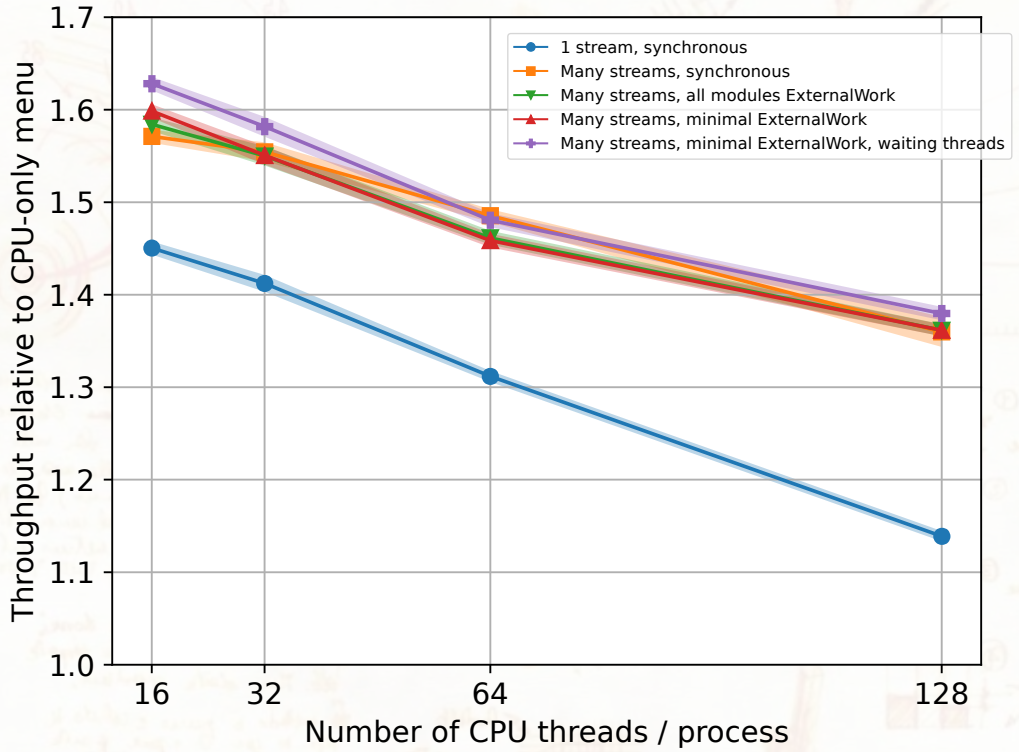
- **caching and reuse of queues and events**
  - *e.g.* CUDA streams, CUDA events
  - avoid the expensive construction and destruction of the underlying objects
- **caching allocator for host memory**
  - support pinned host memory with direct access and queue-ordered delayed deallocations
- **caching allocator for device memory**
  - inspired by CUB's `cub::CachingDeviceAllocator`
  - support global device memory with queue-ordered access
  - implement queue-ordered, asynchronous semantics also for backends that do not support it natively (like SYCL)
- **asynchronous device memory allocations**
  - *e.g.* `hip/cudaMallocAsync()`, `hip/cudaFreeAsync()`
  - available for ROCm ≥ 5.6, CUDA ≥ 11.2
  - alpaka implementation for CPUs
- **synchronous host and device memory allocations**



Impact of different optimisations on the 2024 HLT CMSSW application, offloading part of the reconstruction to one NVIDIA L4 GPU.

CMSSW asynchronous execution

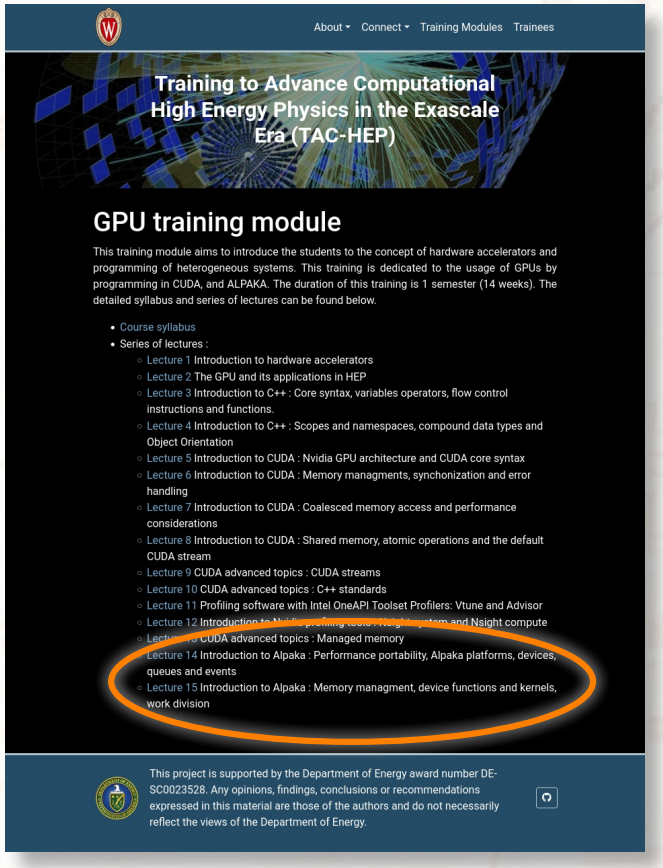
- how to achieve efficient offloading to GPUs ?
- exploit **stream-based parallelism**
  - each group of algorithms enqueues GPU operations to an independent GPU stream
  - each concurrent event has its own streams
- baseline approach: **synchronous operations**
- optimised approach: **asynchronous execution**
  - dedicated pool of sleep-waiting threads
  - automatic on-demand copies to GPUs and host
- the drop in performance as the number of threads increases is due to contention of the internal CUDA mutex
  - options to reduce this contention are under being investigated

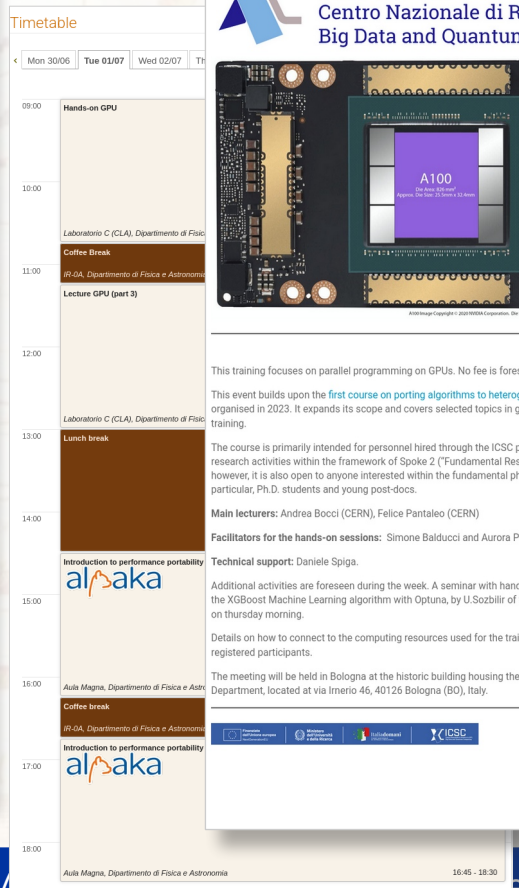


Impact of different GPU submission and synchronisation mechanism on the 2022 HLT CMSSW application, offloading part of the reconstruction to two NVIDIA T4 GPUs.

outreach

tutorials and workshops









CC BY SA

hackathons

- “Patatrack” hackathons at CERN
  - 2–3 times per year, hosted at CERN IdeaSquare
  - one-offs: Bologna (2025), Lyon (2026)
  - topics span heterogeneous and distributed computing, performance portability, machine learning, R&D, …
- Next one, dedicated to DPG activities !
  - 22nd Patatrack Hackathon
  - CERN IdeaSquare, 26-30 October
- GPU hackathons in Europe
  - CNRS / IDRIS
  - Helmholtz / Jülich





The alpaka C++ library for *performance portability*

28th Conference on Computing in High Energy and Nuclear Physics

Abdulrahman Almarzouqi[1], Andrea Bocci[2], Anton Reinhard[3,4], Aurora Perego[2,5], Bernhard Manfred Gruber[2,3,4], Julian Lenz[3,4], Lorenzo Moneta[2], Maria Michailidi[2], Mehmet Yusufoglu[3,4], René Widera[4], Ryan Cross[6], Sanjiban Sengupta[2,7], Simeon Ehrig[4], Simone Balducci[2,8,9], Stephen Nicholas Swatman[2], Tapish Narwal[3,4], Tim Hanel[4,10]

[1] University of Bahrain, [2] CERN, [3] CASUS, [4] Helmholtz-Zentrum Dresden-Rossendorf, [5] University of Milano Bicocca, [6] University of Warwick, [7] University of Manchester, [8] University of Bologna, [9] INFN-CNAF, [10] TU Dresden



CMS tutorials about GPUs and alpaka

- Latest tutorials
  - **Heterogeneous programming with Alpaka** (February 2024)
    - part 1: Introduction to GPUs
    - part 2: Performance portability with alpaka and efficient data structures
    - part 3: Writing kernels and portable modules with CMSSW
  - **An introduction to alpaka** (November 2024)
  - **An introduction to GPU programming in CMSSW** (March 2025)
- Upcoming tutorials
  - **An introduction to GPU programming in CMSSW** (26th of October 2026, 22nd Patatrack Hackathon)
  - **a week-long training event** (early 2027, aiming for late January or February)
    - in collaboration with Trigger Coordination and Offline Software & Computing
    - 2 days dedicated to more general topics: CMSSW, GPUs and heterogeneous programming
    - 3 days dedicated to hands-on on heterogeneous programming with Alpaka in CMSSW

CC BY SA

An introduction to GPU programming in CMSSW

- how to write GPU-friendly data structures
  - Structure of Arrays, PortableObjects, PortableCollections
  - how to automate the copies to and from GPU memory
- how to write and use EDProducers that can run algorithms on the GPUs
  - global vs stream
  - asynchronous vs synchronising
  - making configuration parameters available on the GPU
  - simple introduction to writing kernels
- how to write and use conditions data and ESProducers for GPUs
  - writing conditions data
  - writing and using ESProducers
- how to write EDAnalyzers to look at data produced on the GPUs
- putting it all together and running a CMSSW job

just a table of contents
come to the hackathon !

CC BY SA

the future: alpaka 3 ?

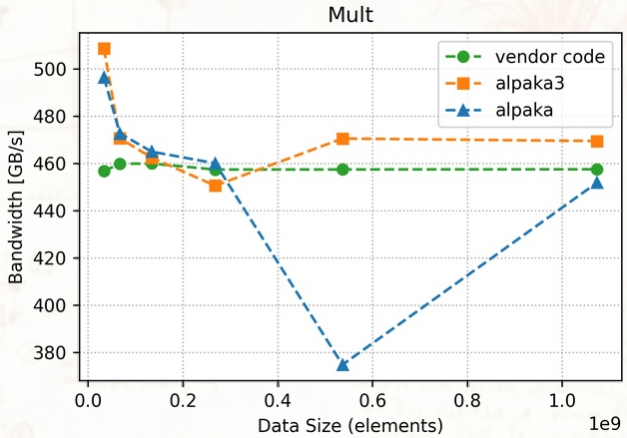
alpaka development history



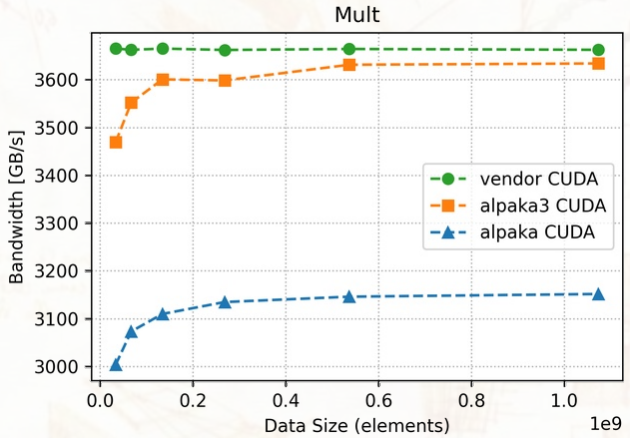
- alpaka 1.0.0 released on November 2023
  - experimental support for Intel oneAPI, with SYCL Unified Shared Memory model
  - support c++23 `std::mdspan` and Kokkos' `mdspan`
- alpaka 1.1.0 released on January 2024
  - stable support for Intel oneAPI
  - implement additional math functions and warp-level functions
  - HZB started using alpaka for RAYX
  - CMS migrates from CUDA to alpaka
- alpaka 1.2.0 released on October 2024
  - more complete support for Intel oneAPI
  - introduce helpers for writing parallel kernels
  - alpaka^3 development started
- alpaka 1.3.0 released on June 2025
  - bug fix release for long term support
  - stable branch with support for c++17
- alpaka 2.0.0 released on June 2025
  - move to c++20 and introduce Concepts
  - make more device-side operations `constexpr`
  - improve memory buffers and views
  - improve support for Intel oneAPI and Altera FPGA
- alpaka 2.1.1 released on December 2025
  - implement STL-like `alpaka::transform`
  - support unified memory
  - support NVIDIA and AMD GPUs in the SYCL backend
  - support for "adopting" non-alpaka GPU streams
  - more CERN projects start using alpaka
- alpaka 2.2.0 just released
  - implement warp mask functions
  - support grid-wide synchronisation
  - add memory fences and improve atomic operations
  - add an interface for external memory allocators
  - support the latest GPUs and frameworks

- new alpaka major version, to be released around the end of 2026
  - improved usability
  - better performance
  - easier coding
  - full SIMD support

BabelStream benchmark



NVIDIA GH200



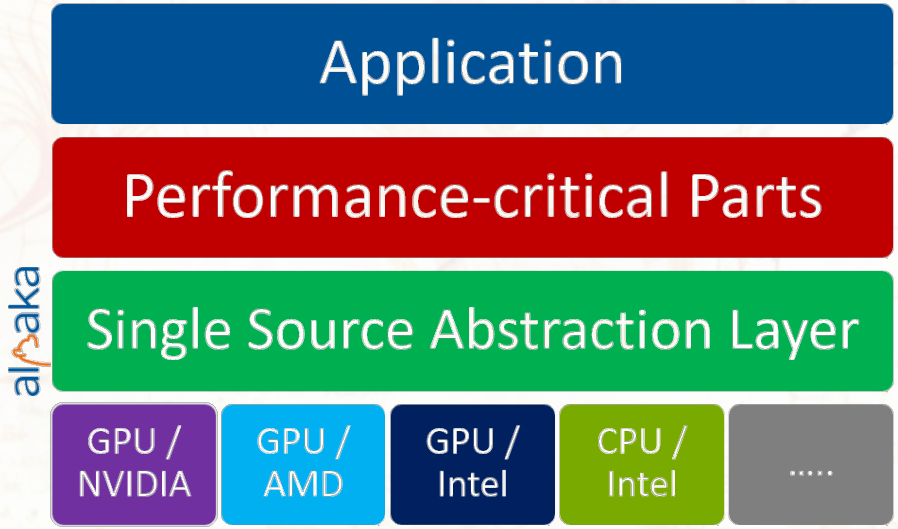
NVIDIA Grace CPU (OpenMP)

- alpaka 3.0
  - in feature freeze
  - release by the end of 2026
- alpaka 3.1
  - include a compatibility library for alpaka 2.x
  - release in early 2027

https://github.com/alpaka-group/alpaka3

alpaka 3 highlights

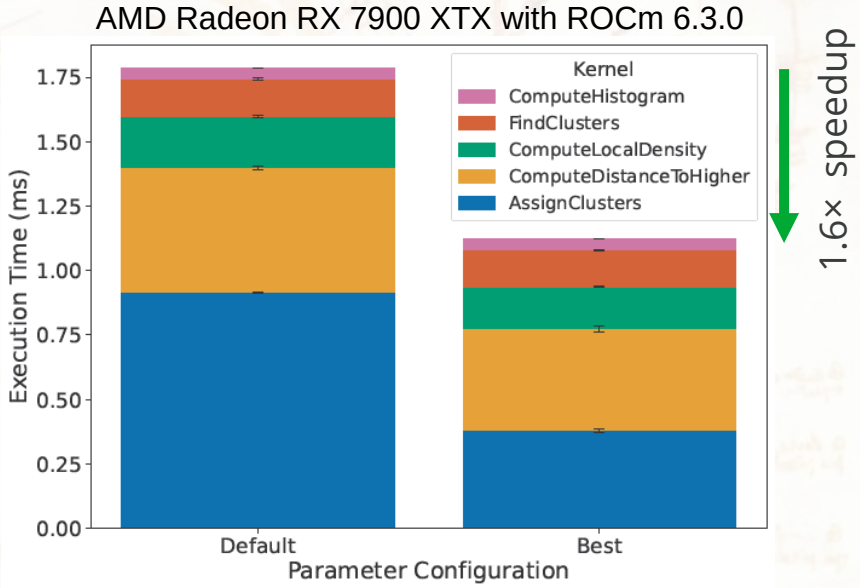
- beginner friendly, with simplified interface
  - syntax feels more “pythonic”
  - separate “onHost” and “onDevice” functions
- safe interface using C++20 concepts
- better performance
- more powerful kernel language
  - group of workers mapped to different problem domains
  - support for compile-time size of blocks and grids
- SIMD support
  - unary SIMD math functions
  - algorithms: iota, scan, transform, reduce, concurrent, …
- function dispatch to vendor APIs
- official support for runtime back-end selection
- improved error logging with dynamic or static log levels



☺ High maintainability
☺ Only critical parts experts needed
☺ High portability
☺ High code quality

CC BY SA

alpaka 3 Auto Tuner – CLUE algorithm



- optimise kernel parameters at runtime
- best configuration can be stored for re-use

standard code

```
queue.enqueue(exec, frameSpec, Kernel{}, arg0);
```

auto tuner

```
auto blockSizes = Tunable<ID1>(
    Vec{8, 4}, Vec{32, 16}, Vec{4, 4});
auto session = TuningBuilder{}.
    withStrategy(strategy::randomSample).
    withThreadBlockSizeTune(blockSizes).build();
session.enqueue(queue, exec, frameSpec, Kernel{}, arg0);
```

questions ?